# DeFenzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Anonymous Author(s)

**Abstract— Compiler-instrumented defenses (e.g., stack canaries, Control-Flow Integrity (CFI), and `_FORTIFY_SOURCE`) form a last line of protection against the exploitation of software flaws, such as memory corruption, control-flow hijacking, and data leakage. However, these defenses can fail silently: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. For example, in CVE-2023-4039, dynamic stack allocation allowed an overflow to bypass the GCC stack canary on AArch64. Existing compiler testing tools fail to detect such bugs. Conventional crash and differential testing only check whether a program produces correct outputs, leaving a fundamental oracle gap when a defense is silently weakened or broken.**

**In this paper, we present DEFENZZ (Defense Fuzzer), an invariant-directed agentic framework for discovering silent failures in compiler-instrumented defenses. DEFENZZ first extracts security invariants from compiler source code, official compiler and ISA documentation, and historical bug fixes using a hybrid Chain-of-Thought (CoT) and Retrieval-Augmented Generation (RAG) pipeline. It then maps these invariants to compiler passes and decision branches, locating where defenses are implemented and controlled. Here, a defense violation occurs when a compiled program fails to preserve the required protection rule. To detect violations reliably, DEFENZZ formalizes these invariants in a predicate IR and lowers them into programmatic bug oracles, implementing them as static and dynamic checkers across target ISAs. All bug verdicts are decided strictly by these checkers, with zero LLM involvement in bug adjudication. Guided by the mapped compiler locations, specialized LLM agents perform directed fuzzing to hit targeted defense branches, synthesizing and mutating seeds to trigger invariant violations under oracle evaluation. We evaluated DEFENZZ on GCC and LLVM across multiple ISAs. DEFENZZ discovered 111 previously unknown bugs in compiler-instrumented defenses, of which 30 have been officially confirmed upstream, including 11 fixed. Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFENZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.**

## I. INTRODUCTION

Compilers deploy a wide range of compiler-instrumented defenses to harden C and C++ systems software against the exploitation of unknown or unpatched vulnerabilities, such as memory corruption, control-flow hijacking, and sensitive data leakage. During compilation, these defenses insert runtime bounds checks, enforce stack layout rules, or clear registers to block exploit primitives. However, implementing these defenses accurately inside production compilers like GCC and LLVM is difficult. Defense code must interact with complex optimization passes and translate abstract protection rules into machine instructions across diverse target ISAs. This translation relies on strict assumptions about object bounds, instruction semantics, register conventions, and stack frame layouts. When the implementation of a defense mechanism fails to match these ISA or optimization assumptions, its security protection breaks from within. At the language level, compiler optimizations assume programs are safe and free of undefined behavior (UB), whereas mitigations assume these properties are violated at runtime. Crucially, the compiler does not distinguish between security checks and regular code, so there is no explicit barrier protecting them. Mitigation passes try to retrofit these checks into the intermediate representation, but modeling them as explicit barriers across the entire compiler pipeline would be too expensive in lost optimizations and rewriting costs. Consequently, subsequent optimization passes can unwittingly weaken or eliminate placed mitigations.

When an optimization relaxes or removes a check or backend lowering violates an ISA rule, it often causes **silent defense failures**: the compiler produces a working binary without any warnings or crashes, yet the security protection is silently weakened or broken. Because the lack of a mitigation is only detected when the program executes outside standard language semantics, regular executions on benign inputs run normally, allowing standard functional tests to pass completely while leaving the software exposed to attack. For example, in **CVE-2023-4039** [1], GCC on AArch64 placed the saved frame pointer and return address at the bottom of the fixed stack frame, below the stack canary. When a function used `alloca` or a variable-length array, GCC allocated that dynamic buffer at a lower stack address right below the saved return address. An upward buffer overflow from the dynamic region could then overwrite the saved return address without ever touching the canary.

Because modeling explicit barriers across the compiler pipeline is impractical, our best bet is to check the correctness of placed mitigations after compilation. Existing compiler testing approaches, however, fail to catch such silent bugs due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers (e.g., Csmith [2] and YARPGen [3]) check for compiler crashes or output mismatches across optimization levels. Because a binary with a broken defense still produces correct outputs on benign inputs, crash and differential oracles remain blind when a defense is weakened or broken. The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can reach given target branches, but they cannot tell which branches inside millions of lines of compiler code control a defense. Even after reaching a defense branch, exposing a silent failure requires satisfying

defense-specific semantic conditions, such as opaque object bounds or cross-pass length propagation, rather than generic control-flow or data-flow coverage. For instance, in a silent `_FORTIFY_SOURCE` bug in GCC [4], triggering relaxation in the `strlen` pass requires pairing `__strcat_chk(dst, src, objsize)` on an unknown-length buffer with a subsequent `strlen(dst)` call that triggers the rewrite into `__stpcpy_chk`. Random mutators rarely synthesize such coupled defense semantics. Meanwhile, building static analyzers demands heavy manual effort from experts to craft custom rules for each defense mechanism and target ISA.

These barriers raise a central question: how can we systematically verify placed mitigations after compilation when the buggy compiler itself cannot serve as a reliable oracle? To answer this question, we present **DEFENZZ** (**Defen**se Fu**zz**er), an invariant-directed agentic framework for testing compiler-instrumented defenses. Our core insight is that although defense implementations vary across backends, their intended security guarantees follow target-independent *security invariants*, where any compiled binary breaking a required rule constitutes a defense violation. However, manually extracting invariants and building oracles across dozens of defenses, massive compiler codebases, and complex ISA manuals is impractical. DEFENZZ therefore uses LLMs to extract security invariants from compiler code, documentation, and historical patches. It then applies LLMs to translate these invariants into programmatic bug oracles implemented as ISA-specific static and dynamic checkers, closing the oracle gap. Because LLM-assisted translation can introduce errors, DEFENZZ formalizes each invariant in a predicate IR with explicit applicability and safety semantics, deterministically lowering validated specifications into executable checkers. To trigger defense-critical logic, DEFENZZ maps each invariant to its corresponding defense-related compiler branches and guides LLM agents to synthesize inputs that satisfy both branch conditions and invariant preconditions. All bug verdicts remain strictly with deterministic checkers, avoiding both the blind spots of random mutators and LLM adjudication hallucinations.

Building DEFENZZ requires solving three main challenges. The first challenge is defining and establishing reliable security invariants: because the compiler itself has bugs, we cannot trust it. Relying on a compiler's own behavior or implementation code as the ground truth creates circular self-validation. Our key idea is to define machine-checkable security contracts that capture intended protection rules independently of buggy compiler implementations, anchoring invariants in external ISA manuals, documented design assertions, and confirmed security patches, while abstracting past fixes into transferable failure patterns across defenses. The second challenge is checking these invariants with high accuracy across diverse ISAs, where distinct registers, calling conventions, and stack layouts make manual rule authoring and heuristic checks error-prone. Meanwhile, using an LLM directly as a bug oracle suffers from severe hallucinations on low-level ISA details and is too slow for fuzzing. Reliable fuzzing thus requires deterministic oracles, yet manual implementation cannot scale

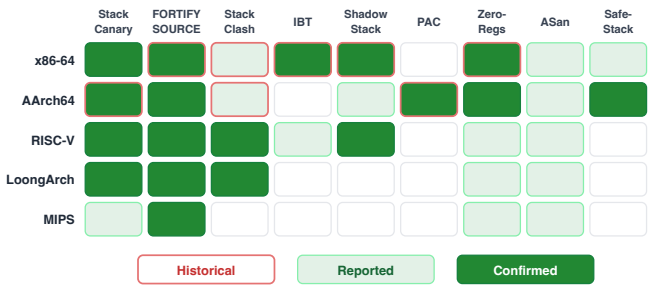



Fig. 1. Partial matrix of silent compiler defense bugs across representative defense mechanisms and target ISAs.

and direct LLM translation into checkers still causes hallucinations. DEFENZZ addresses this by formalizing binary and runtime query primitives in Check IR alongside target ISA specifications. The LLM then writes high-level check logic to combine these primitives and ISA parameters into either a static or dynamic checker. During fuzzing, these deterministic checkers directly test each compiled seed, rather than relying on an LLM to judge whether silent defense bugs occur. The final challenge is steering test generation to both locate defense branches and satisfy their semantic preconditions without being stripped by early optimizations. DEFENZZ maps each invariant to its controlling compiler passes and branches via call-graph and AST analysis, inserting compile-time probes to trace branch execution. Guided by branch traces and invariant preconditions, LLM agents iteratively synthesize and mutate seeds to expose deep defense flaws.

We evaluated DEFENZZ on GCC 16.2.0 and LLVM 23.1.2 across x86-64, AArch64, and RISC-V, spanning diverse defenses such as stack canaries, CFI, shadow stacks, register scrubbing, and `_FORTIFY_SOURCE`. DEFENZZ extracted 915 validated security invariants and transferred historical failure patterns across defense mechanisms and backends. During fuzzing, branch-guided seed synthesis hit substantially more target defense branches than unguided baselines, while our formally specified checkers identified violations with high accuracy and avoided the false reports produced by unconstrained LLM agents. Overall, DEFENZZ discovered **111 previously unknown silent failures in compiler-instrumented defenses**, of which **30 have been officially confirmed upstream, including 11 fixed** (Figure 1), including bugs rated as high-severity vulnerabilities. These results show that DEFENZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

In answering this central question, this paper makes the following contributions:

- **Security Invariant Definition and Extraction:** We define target-independent security invariants and decouple security contracts from buggy compiler outputs by extracting invariants from ISA specifications, design assertions, and historical bug fixes, transferring failure patterns across defenses.
- **Validated Translation into Multi-ISA Programmatic**

**Oracles:** We formalize security invariants in a predicate IR and lower them into formally specified static and dynamic checkers across target ISAs, verifying compiled code with high accuracy and zero LLM involvement in bug adjudication.

- **Branch-Guided Agentic Fuzzing:** We map invariants to defense-related compiler passes and branches, and guide specialized LLM agents with compile-time branch traces to synthesize seeds that satisfy defense triggering conditions.
- **New Silent Defense Bug Discovery:** We evaluate DEFENZZ on GCC 16.2.0 and LLVM 23.1.2 across multiple ISAs and discover 111 previously unknown silent failures in compiler-instrumented defenses, with 30 officially confirmed upstream, including 11 fixed.

## II. MOTIVATION

### A. Compiler-Instrumented Defenses

Compilers deploy automated defenses to protect software against security threats. Unlike standalone binary rewriters, compiler-instrumented defenses span both middle-end and backend stages of compilation. Middle-end passes analyze object bounds and control flow to insert or optimize high-level checks on the intermediate representation (IR). Backend passes enforce low-level protections by controlling stack frame layouts, register assignments, and machine instructions across target ISAs.

Preserving security guarantees across this multi-stage pipeline is error-prone. A middle-end optimization can rewrite a defense check under flawed size assumptions, or a backend pass can misplace a stack guard slot. When such bugs occur, the compiler neither crashes nor emits warnings. Instead, it produces a working binary whose security protection is quietly weakened or broken.

### B. Motivating Example

Listing 1 shows a silent defense failure in GCC's `_FORTIFY_SOURCE` implementation. This bug had remained hidden for 14 years before upstream maintainers fixed it in the GCC mainline. Under `-D_FORTIFY_SOURCE=2`, GCC replaces unsafe string calls such as `strcat(dst, src)` on the 64-byte buffer `dst` with the bounds-checked builtin `__strcat_chk(dst, src, 64)`. At runtime, this builtin checks that the existing string in `dst` plus the appended `src` fits within 64 bytes, aborting execution on overflow.

```
1  char dst[64];
2
3  size_t append_and_measure(const char *src) {
4    init_opaque(dst); // hides prefix len k (k = 8)
5
6    // 1. Fortified call: __strcat_chk(dst, src,
       64)
7    strcat(dst, src);
8
9    // 2. get_string_length() rewrites call to:
10   // __stpcpy_chk(dst + k, src, 64)
11   // BUG: bound stays 64 instead of 64 - k!
12   return strlen(dst);
13 }
```

Listing 1. Silent `_FORTIFY_SOURCE` relaxation in GCC.

In Listing 1, `init_opaque(dst)` initializes the 64-byte buffer `dst` with an 8-byte string ($k = 8$) whose length is unknown at compile time, followed by `strcat(dst, src)` and `strlen(dst)`. Executing `strcat(dst, src)` and `strlen(dst)` naively would scan `dst` twice: once to find the end of the prefix, and again to measure the combined string. To avoid this redundant scan, GCC's `tree-ssa-strlen` pass invokes `get_string_length()` in `tree-ssa-strlen.cc` when it processes the trailing `strlen(dst)` call. Inside `get_string_length()`, the `BUILT_IN_STRCAT_CHK` branch rewrites the earlier `__strcat_chk(dst, src, 64)` into `__stpcpy_chk(dst + k, src, 64)`, which copies `src` to `dst + k` and returns the end pointer to compute `strlen(dst)` directly. However, while GCC advances the destination pointer by $k$ bytes, it fails to subtract $k$ from the size bound, passing 64 instead of $64 - k = 56$. As a result, the runtime check still permits writing 64 bytes starting from `dst + 8`, allowing an 8-byte buffer overflow without triggering an abort.

### C. Limitations of Existing Approaches

Existing compiler testing tools fail to detect this vulnerability due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers such as Csmith [2], YARPGen [3], and EMI [5] only check whether the compiler crashes or whether compiled binaries produce consistent outputs on benign inputs. In Listing 1, GCC compiles the code cleanly, and any benign string `src` produces the exact same return value across all optimization levels. Because crash and differential testing do not check whether the binary enforces the reduced bound $(64 - k)$, they remain blind when the defense is weakened.

The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can steer execution toward given target branches, but they cannot tell which branches inside millions of lines of compiler source code control a defense, such as the `BUILT_IN_STRCAT_CHK` branch in `get_string_length()`. Even after reaching that branch, triggering the bug in Listing 1 requires three coupled conditions on `dst`: the buffer must have a known allocation size so GCC emits `__strcat_chk(dst, src, 64)`, its prefix

length $k$ must remain unknown at the append site so folding is deferred, and a subsequent `strlen(dst)` call must query the length to trigger the rewrite into `__stpcpy_chk(dst + k, src, 64)`. Random mutators rarely chain an opaque initialization, a fortified append, and a subsequent `strlen` call on the same buffer.

Beyond fuzzing, static analysis requires heavy manual effort from human experts to craft custom rules for each defense mechanism and target ISA. Meanwhile, using LLMs to audit compiler source code end-to-end causes severe hallucinations and false bug reports. To overcome these limitations, a testing framework must locate defense-related compiler branches and guide seed generation toward them. The generated seeds must satisfy the branch conditions so the compiler emits the defense code in the binary. The framework must then run the compiled binary with defense-triggering inputs and use deterministic oracles to check whether the protection is silently weakened or broken.

## III. Design of DeFenzz

In this section, we present the design of DeFenzz, an invariant-guided framework for discovering silent failures in compiler-instrumented defenses. Figure 2 illustrates its overall workflow, which consists of four core steps: security invariant generation, compiler source mapping, programmatic oracle construction, and agentic directed fuzzing. First, we extract security invariants from compiler source code, official documentation, and historical bug reports using a hybrid CoT and RAG pipeline. Second, we map these invariants to the relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. Third, we combine Check IR primitives with target ISA specifications to synthesize executable static and dynamic Go checkers across architectures, verifying whether compiled programs preserve intended defenses without model hallucination. Finally, our agentic directed fuzzing coordinates a generator agent and a feedback agent to iteratively synthesize and mutate test programs under branch-trace and checker feedback. We discuss each step in detail in the following subsections.

### A. Security Invariant Generation

To detect silent defense failures, DeFenzz extracts security invariants that specify the protection rules each compiler defense must enforce. Because the compiler itself may contain bugs, neither its implementation logic nor its compiled outputs can serve as the ground truth. Treating current compiler behavior as the reference would cause circular self-validation whenever a bug silently weakens or breaks a defense. Instead, DeFenzz extracts security invariants from three sources: official compiler documentation and ISA/ABI manuals, developer comments and assertions in compiler source code, and confirmed historical bug reports.

Feeding whole compiler files or manuals to an LLM exceeds context limits and causes hallucinations. To keep each prompt focused, DeFenzz splits compiler source code and official documentation into small, self-contained slices. For compiler source code, each slice contains a single C/C++ function with its leading header comment, or a single machine-description pattern (`define_insn` or `define_expand`). For official documentation, each slice captures a self-contained manual section defining defense requirements, such as a command-line option entry in GCC's user manual, a target-hook specification in GCC's internals manual, or a feature design section in Clang documentation. Every slice records its source path, line boundaries, defense mechanism, and target ISA. Because these slices are completely independent, DeFenzz analyzes them in parallel using CoT prompting [6] to extract the triggering precondition, the protection rule, and the observable violation signal. In the fortified string bug, reviewing the slice for `get_string_length()` and its header comment leads the model to infer a clear rule: when advancing a destination pointer by a prefix length, the pass must subtract that same length from the remaining size bound.

Different defenses and backends often perform similar low-level operations, such as updating size bounds, laying out stack frames, or clearing registers before return. As a result, new silent failures often share the same root causes as historical bugs. To find such bugs across defenses and architectures, DeFenzz uses a hybrid RAG pipeline [7], [8] to transfer historical bug patterns. Searching directly with a raw bug patch only matches the original buggy function. Instead, DeFenzz strips defense-specific function and flag names from each historical bug report, distilling a general query with three parts: a root-cause summary, defense-agnostic action keywords, and original defense terms to exclude, as shown in Listing 2 for GCC PR 96191 [9]. Using this query, DeFenzz searches the sliced source and documentation corpus by combining BM25 keyword search with dense vector retrieval via Reciprocal Rank Fusion. BM25 matches underscore-split identifier tokens that share naming conventions across passes, such as `ix86_zero_call_used_regs`. Dense vector retrieval finds slices that implement the same operation with different data structures, such as the register bitmap in ARM's `cmse_nonsecure_entry_clear_before_return`. After filtering out slices that contain the excluded terms, DeFenzz selects the top-$k$ candidate slices. For each candidate slice, the model pairs the code with the historical root cause and uses CoT reasoning to derive a new invariant for that target defense.

```
# Distilled Query from GCC PR 96191
root_cause: "Function exit path returns without
             clearing caller-visible registers"
tokens:     ["epilogue", "caller_saved", "clear",
             "register", "return"]
exclude:    ["stack-protector", "canary"]
# Top Cross-Defense Hits via BM25 + Dense RRF
hit_1_bm25:  "config/i386/i386.cc"
  fn:  "ix86_zero_call_used_regs"
  why: "sub-tokens {zero, call, regs}"
hit_2_dense: "config/arm/arm.cc"
  fn:  "cmse_nonsecure_entry_clear_before_return"
  why: "root cause on register-clear bitmap"
```

Listing 2. RAG query structure and cross-defense retrieval hits.

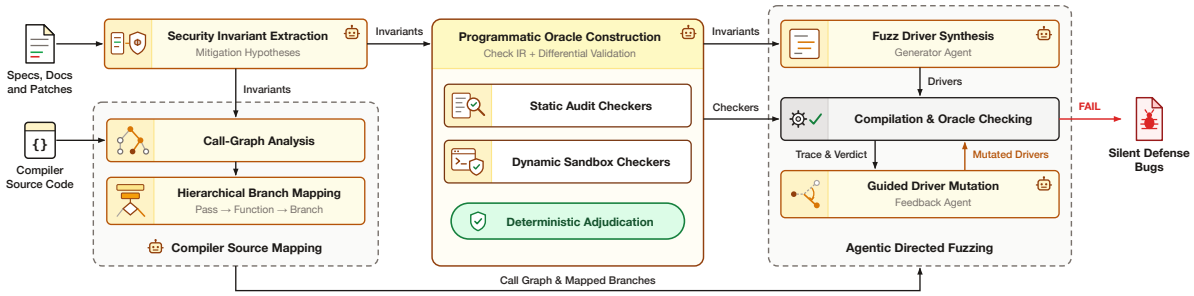



Fig. 2. Overview of DEFENZZ across four stages: (1) extracting security invariants via CoT and RAG, (2) mapping invariants to compiler branches, (3) combining Check IR primitives and target ISA specifications into multi-ISA checkers, and (4) branch-guided agentic fuzzing.

All invariants from CoT analysis and RAG transfer are stored as structured records in a unified catalog, as shown in Listing 3. Each record tags the target defense and applicable ISAs, followed by three core fields: the triggering precondition, the protection rule, and the observable violation signal. Because these records describe security rules without tying them to specific compiler files or functions, the same invariant can guide source mapping across different compilers and build binary checkers across multiple ISAs.

```
id:       "INV-FORT-014"
defense:  "_FORTIFY_SOURCE"
isas:     ["x86_64", "aarch64", "riscv64"]
precond:  "Fortified append on sized buffer N with
           prefix len k, followed by length query"
rule:     "Rewriting append to offset dst + k must
           reduce remaining size bound to N - k"
signal:   "Bound argument stays > N - k, or runtime
           write past N bytes fails to abort"
```

Listing 3. Structured security invariant record in the catalog.

### B. Compiler Source Mapping

With the invariant catalog in place, DEFENZZ maps each security invariant to the compiler passes and decision branches that implement the target defense. An invariant defines what rule a compiled program must follow, but not where the compiler enforces that rule across millions of lines of C and C++ code. Simple keyword search is not enough because a single defense often spans middle-end passes, target hooks, and backend machine descriptions. To navigate the codebase accurately, DEFENZZ gives a mapping agent three static analysis tools: a symbol indexer, a call-graph analyzer, and an AST parser. Using these tools, the agent locates defense logic along a three-level hierarchy of passes, functions, and branches.

Starting from defense flags or builtin names, the agent queries the symbol indexer and call graph to find the pass that implements the defense. Within that pass, it follows call-graph edges to find functions that compute object bounds, lay out stack frames, or insert runtime checks. It then parses the AST of each function to extract the conditional branches and variables that control the defense. Listing 4 shows the resulting mapping for `INV-FORT-014`, which traces fortified string calls to the `tree-ssa-strlen` pass, the `get_string_length()` function, and the `BUILT_IN_STRCAT_CHK` branch.

```
# 1. Three-Level Hierarchy Mapping (INV-FORT-014)
pass:     "tree-ssa-strlen"
file:     "gcc/tree-ssa-strlen.cc"
function: "get_string_length"
branch:   "case BUILT_IN_STRCAT_CHK"
  vars:   ["elen", "objsize", "lhs"]
  log_id: "fortify#b4e1"
# 2. Instrumented Branch in get_string_length()
case BUILT_IN_STRCAT_CHK:
  DEFENZZ_HIT("fortify#b4e1:hit");
  if (elen != NULL && objsize != NULL) { ... }
# 3. Emitted Compile-Time Log
[DEFENZZ_LOG] fortify#b4e1:hit
```

Listing 4. Source mapping and branch logging for `INV-FORT-014`.

Based on each mapping, DEFENZZ assigns a unique branch tag such as `fortify#b4e1`, where `b4e1` hashes the mapped file, line, and branch condition, and inserts a logging probe right before the branch, as shown in Listing 4. When the instrumented compiler builds a test seed, each reached probe prints its branch tag to the compile-time log, showing the fuzzer which mapped defense branches were executed during compilation and which remained unreached.

### C. Programmatic Oracle Construction

To check whether a test program $p$ preserves a defense, DEFENZZ turns each security invariant into an executable Go checker across target ISAs, with **zero LLM involvement in bug adjudication**. However, asking an LLM to write raw checkers from scratch is unreliable. Checking a security invariant requires combining high-level verification logic with low-level ISA and ABI rules, such as register roles, calling conventions, and stack layouts. Relying on the model's internal knowledge for these low-level rules easily causes hallucinations, especially on less common architectures. Moreover, writing raw checkers from scratch—parsing compiler dumps, assembly text, ELF sections, and runtime outputs directly for each invariant—often introduces coding bugs, causing false alarms or missed bugs.

DEFENZZ addresses both issues through a **target specification table** $\mathcal{T}_i$ and a library of **Check IR primitives** $\Omega$.

Built from official ISA and ABI manuals, $\mathcal{T}_i$ stores exact parameters for each architecture $i$, including stack growth direction, return-address location, caller-saved and argument registers, CFI landing-pad bytes, and instruction alignment, as shown in Table I. Meanwhile, Check IR provides built-in primitives $\Omega$ that query individual facts from $p$ across its compiler dumps, assembly code, ELF binary, and runtime execution. Static primitives in $\Omega$ inspect dumps, assembly, and ELF sections to query imported symbols, call sites, pointer offsets, and constant call arguments. Dynamic primitives in $\Omega$ run $p$ to observe runtime behavior, such as exit signals, output markers, and post-return register values.

Rather than encoding an entire checker as a single rigid expression or writing raw checkers from scratch, the LLM writes high-level Go logic over $\Omega$ and $\mathcal{T}_i$ to form a complete checker $\phi$:

$$\phi = \langle \mu,\ I_\phi,\ G_{\text{app}}(p, \mathcal{T}_i),\ P_{\text{safe}}(p, \mathcal{T}_i) \rangle. \tag{1}$$

Each checker $\phi$ declares a single checking mode $\mu \in \{\text{Static}, \text{Dynamic}\}$ and its applicable architectures $I_\phi$. Using Go control flow over primitives in $\Omega$ and fields in $\mathcal{T}_i$, $\phi$ implements two boolean checks: a precondition guard $G_{\text{app}}(p, \mathcal{T}_i)$ that tests whether $p$ activates the target defense, and a safety check $P_{\text{safe}}(p, \mathcal{T}_i)$ that tests whether the protection holds. Given a test program $p$ on target $i \in I_\phi$, running $\phi$ returns one of four verdicts:

$$\text{Verdict}(p, \phi, i) = \begin{cases} \textbf{ERROR}, & \text{if build or query fails,} \\ \textbf{NA}, & \text{if } \neg G_{\text{app}}, \\ \textbf{PASS}, & \text{if } G_{\text{app}} \wedge P_{\text{safe}}, \\ \textbf{FAIL}, & \text{if } G_{\text{app}} \wedge \neg P_{\text{safe}}. \end{cases} \tag{2}$$

Build or query failures return **ERROR** instead of false bug alerts, and programs that do not trigger the defense return **NA** instead of false passes.

Listing 5 shows how a static Go checker combines Check IR primitives and $\mathcal{T}_i$ for `INV-FORT-014`. For $G_{\text{app}}$, the checker calls `ir.CallSites` to locate rewritten `__stpcpy_chk` calls in $p$, returning **NA** when none exist. For $P_{\text{safe}}$, the LLM writes a loop over each call site. Inside the loop, primitives query the destination buffer size and check whether the destination pointer in the first argument register advances past the buffer base. The checker then reads the constant bound loaded into the third argument register. When GCC advances the destination pointer by $k$ bytes but keeps the bound at 64 bytes, the bound exceeds the remaining space and the checker returns **FAIL**.

When an invariant depends on runtime state, a dynamic Go checker invokes execution primitives in $\Omega$ to run $p$. In GCC PR 96191 [9], a function checks its stack canary in a scratch register but fails to clear that register before returning. Its dynamic checker verifies that $p$ imports `__stack_chk_fail` for $G_{\text{app}}$, calls `ir.PostReturnRegs` to run $p$ and record the caller-saved registers in $\mathcal{T}_i.R_{\text{caller}}$ after return, and loops over those registers to return **FAIL** if any register still holds the canary.

TABLE I
SELECTED ISA AND ABI PARAMETERS IN THE TARGET SPECIFICATION TABLE $\mathcal{T}_i$.

| Field in $\mathcal{T}_i$ | x86-64 | AArch64 | RISC-V |
|---|---|---|---|
| $\delta_{\text{grow}}$ (stack) | $-1$ (down) | $-1$ (down) | $-1$ (down) |
| ra (ret addr) | Stack slot | Reg `x30` | Reg `ra` |
| $R_{\text{caller}}$ (regs) | `rax,rcx..rdi,r8–r11` | `x0–x18` | `t0–t6, a0–a7` |
| $R_{\text{arg}}$ (arg regs) | `rdi,rsi,rdx,rcx...` | `x0–x7` | `a0–a7` |
| pad (CFI pad) | `f30f1efa (endbr64)` | `5f2403d5 (bti)` | `17000000 (lpad)` |
| align (step) | 1 byte | 4 bytes | 2 bytes |

```
func CheckFortify(p *Prog, T Target) Verdict {
  // G_app: check rewritten __stpcpy_chk calls
  sites, err := ir.CallSites(p, "__stpcpy_chk")
  if err != nil { return ERROR }
  if len(sites) == 0 { return NA }

  // P_safe: LLM logic over Check IR + T_i
  for _, s := range sites {
    bufSz := ir.DstObjSize(p, s, T.ArgRegs[0])
    moved := ir.HasPtrOffset(p, s, T.ArgRegs[0])
    bnd, ok := ir.ConstArgImm(p, s, T.ArgRegs[2])
    if moved && ok && bnd >= bufSz {
      return FAIL // dst moved, bound >= N
    }
  }
  return PASS
}
```

Listing 5. Static Go checker for `INV-FORT-014` combining Check IR primitives and $\mathcal{T}_i$ with high-level check logic.

**Algorithm 1** Agentic Directed Fuzzing per Invariant

**Require:** Go checker $\phi$, mapped branches $m$, target ISA $i$, budget $B$
**Ensure:** Minimized bug PoCs on shared blackboard
$PoCs \leftarrow \emptyset$; $guide \leftarrow \text{InitGuidance}(\phi, m)$
**for** $t = 1$ **to** $B$ **do**
  $p \leftarrow \text{GeneratorAgent}(\phi, m, guide)$
  $trace \leftarrow \text{CompileInst}(p, i)$
  $verdict \leftarrow \text{Verdict}(p, \phi, i)$
  $\text{RecordBlackboard}(p, trace, verdict)$
  **if** $verdict =$ **FAIL then**
    $p^* \leftarrow \text{MinimizerAgent}(p, \phi, i)$
    $PoCs \leftarrow PoCs \cup \{p^*\} \cup \text{Amplify}(p^*, \phi)$
    **break**
  **else**
    $guide \leftarrow \text{FeedbackAgent}(\phi, m, trace, verdict)$
  **end if**
**end for**
**return** *PoCs*

## D. Agentic Directed Fuzzing

With the invariant catalog, mapped branches $m$, and Go checkers $\phi$ in place, DEFENZZ runs the directed fuzzing loop in Algorithm 1. Letting LLM agents call tools in an open-ended loop often causes aimless exploration and unverified bug claims. Instead, a central orchestrator runs a fixed four-step loop for each invariant and target architecture $i \in I_\phi$ within a budget of $B$ rounds. In each round, a Generator Agent writes a test program $p$, the instrumented compiler compiles $p$ and logs reached branches, the Go checker $\phi$ evaluates $p$, and a Feedback Agent guides the next mutation. The orchestrator records every test program $p$, branch trace, and verdict on a

shared blackboard so that every test case and bug report can be reproduced.

In each round, the Generator Agent writes a C or C++ test program $p$ to reach the mapped branches $m$ and satisfy the precondition $G_{\text{app}}(p, \mathcal{T}_i)$. Triggering a silent defense bug often requires multiple statements to work together on the same buffer. In `INV-FORT-014`, triggering the faulty rewrite requires a fixed-size buffer, an unknown prefix length at compile time, a fortified string append, and a trailing length query on that buffer. Guided by $m$ and the invariant record, the Generator Agent writes small programs that satisfy all of these conditions at once, while hiding selected values across non-inlined helper calls, runtime inputs, and memory barriers so that early optimizations do not fold away the test logic.

When $p$ does not trigger a bug, the Feedback Agent must determine whether compilation failed to reach the target branch $m$ or reached $m$ without violating the invariant $\phi$. To separate these cases, the agent inspects both the compile-time branch trace and the four-state verdict $\mathsf{Verdict}(p, \phi, i)$. A verdict of $\mathbf{ERROR}$ or $\mathbf{NA}$ indicates that $p$ failed to compile or never activated the defense, so the agent fixes syntax errors or adds the missing defense triggers. When $p$ activates the defense and returns $\mathbf{PASS}$, the agent compares the branch trace against $m$: if compilation turned away before $m$, the agent modifies the program constructs that control the missed branch condition; if compilation already reached $m$, the agent mutates the variables and data-flow operands inspected at $m$ to drive the pass into an unsafe corner case.

Once $\phi$ returns $\mathbf{FAIL}$, DEFENZZ uses both $m$ and $\phi$ to minimize and generalize the bug-triggering program $p$. Traditional test-case reducers only preserve compiler crashes or exit codes, which easily collapses a silent defense bug into unrelated undefined behavior during reduction. Instead, the Minimizer Agent pairs C-Reduce [10] with the instrumented compiler and $\phi$, accepting a reduction step only when the candidate program still reaches $m$ and returns $\mathbf{FAIL}$. Because the minimized program $p^*$ isolates the exact flawed compiler logic, the Generator Agent then mutates $p^*$ across sibling builtins, data types, optimization levels, and target ISAs to uncover related silent defense bugs that share the same root cause.

## IV. IMPLEMENTATION

DEFENZZ is implemented as a Python orchestrator paired with a deterministic Go oracle service. In the offline stages, the Python pipeline splits compiler source code and manuals into modular slices, and indexes them for hybrid RAG using dense embeddings fused with BM25. For compiler source mapping, the symbol indexer, call-graph analyzer, and AST parser are built on `clangd`, CodeQL, and Tree-sitter, and exposed to agents as read-only MCP tools. Based on the mapped branches $m$, a deterministic AST rewriter injects logging probes into GCC and LLVM before building the instrumented compilers.

The oracle core is implemented in Go and communicates with the Python orchestrator over gRPC. It hosts the target specification table $\mathcal{T}_i$, the Check IR primitive library $\Omega$, and all compiled Go checkers $\phi$. Static primitives in $\Omega$ inspect compiler pass dumps, assembly files, ELF sections, and multi-architecture disassembly. Dynamic primitives run $p$ inside isolated containers using native execution or QEMU user-mode emulators across target ISAs. During online fuzzing, the Go core also drives C-Reduce [10] for program minimization and records every test program $p$, compile-time branch trace, and checker verdict on a persistent blackboard so that every confirmed bug can be deterministically replayed.

## V. EVALUATION

We evaluate DEFENZZ on GCC and LLVM across multiple target architectures and compiler-instrumented defenses. Our experiments answer four research questions:

- **RQ1 (Bug Discovery):** Can DEFENZZ uncover previously unknown, high-severity silent defense bugs across compilers, defense families, and architectures?
- **RQ2 (Invariant and Oracle Quality):** How effectively does DEFENZZ extract grounded security invariants and reject faulty checkers via offline gates?
- **RQ3 (Ablation Study):** How much does each design choice, starting with formal Check IR synthesis, improve bug detection and precision?
- **RQ4 (Telemetry and Overhead):** What are the agent tool-call patterns, token costs, KV-cache hit rates, and build overheads across pipeline stages?

### A. Experimental Setup

**Target Compilers and Architectures.** We evaluate DEFENZZ on release builds of GCC 16.2.0 (verified against GCC 17.0 trunk) and LLVM/Clang 23.1.2. Each campaign targets four primary 64-bit architectures: x86-64, AArch64, RISC-V (`rv64gc`), and LoongArch64. For portable target hooks, DEFENZZ also cross-compiles seeds for 32-bit x86, ARM, MIPS64, PowerPC64LE, and s390x.

**Evaluated Defense Families.** Table II groups the audited mechanisms into six defense families spanning both middle-end and backend compiler stages. Control-flow defenses cover CET-IBT, ARM BTI, PAC, KCFI, Clang CFI, and RISC-V `Zicfilp`. Stack defenses cover stack canaries, stack-clash probing, SafeStack, CET shadow stacks, ARM GCS, `Zicfiss`, and ShadowCallStack. Memory sanitizers include ASan, HWASan, MTE stack tagging, and UBSan. Scrubbing and speculation defenses include `-fzero-call-used-regs`, `-fstrub`, auto-var-init, SLH, SLS, LVI, and retpolines. The remaining two families cover object-bounds fortification (`_FORTIFY_SOURCE`, `counted_by`, `-fstrict-flex-arrays`, `-fanalyzer`) and call, link, or enclave hardening (RELRO, `noplt`, `noexecstack`, SGX).

**Environment and Model Configuration.** All experiments run inside an isolated Linux container on a 16-vCPU ARM Neoverse-N2 server. The container bundles GNU and LLVM cross-toolchains, QEMU user-mode emulators for x86-64, AArch64, RISC-V, and LoongArch64, and C-Reduce [10].

All LLM agents query `deepseek-flash` with prefix KV-caching enabled. Hybrid RAG indexes 5,380 GCC and 1,030 LLVM slices using `BAAI/bge-small-en-v1.5` embeddings fused with BM25. Step 3 caps offline Check IR repair at three attempts per invariant. Step 4 allocates an online budget of $B = 8$ iterations per invariant and target ISA. All online bug verdicts are decided strictly by compiled checkers, with **zero LLM involvement in bug adjudication**.

**Baselines and Controlled Variants.** We compare DEFENZZ against Csmith [2] and YARPGen [3] compiled under identical hardening flags from `-O0` to `-O3`. We also define five controlled ablations to isolate internal components. *w/o Formal Check IR* replaces Check IR with direct LLM synthesis of raw Go checkers. *w/o Hybrid RAG* disables historical bug transfer, and *w/o Source Mapping* removes AST branch mapping and `DEFENZZ_HIT` probes. *w/o Programmatic Oracle* replaces compiled checkers with LLM judgment, while *Bare Agent* runs an unconstrained tool-using agent.

### B. RQ1: Silent Defense Bug Discovery

**Overall Discovery and Upstream Triage.** Table II summarizes our bug-finding results across GCC and LLVM. DEFENZZ discovered and filed **111 previously unknown silent defense bugs**, comprising 58 in GCC and 53 in LLVM. Among these filed reports, **30 have been officially confirmed upstream (18 GCC, 12 LLVM), including 11 already fixed (9 GCC, 2 LLVM)**, and another 5 carry maintainer wrong-code or miscompilation tags while awaiting formal status transitions. Several fixes were backported to release branches as old as GCC 14. Maintainer review and our pre-filing audit also identified 3 intentional design behaviors, 7 disputed language-boundary cases, 2 independently patched trunk bugs, and 8 candidates with incomplete contract evidence; we exclude all 20 cases from the 111 bug count. Under identical hardening flags, Csmith and YARPGen trigger **[TBD: N]** compiler crashes and functional miscompilations, yet both find **zero** silent defense bugs. Crash and differential testing check output equivalence on benign inputs, leaving them blind when compilers silently weaken or break a defense.

**Stage, Architecture, and Severity Distribution.** Figure 3 characterizes the root causes and affected architectures of all 111 bugs. As shown in Figure 3(a), backend target lowering and frame layout account for 48.6% (54/111) of all bugs, middle-end IR passes account for 22.5% (25/111), and frontend attribute handling accounts for 10.8% (12/111). The remaining 18.0% (20/111) arise in option lowering, IPA/LTO serialization, or runtime libraries; for example, GCC PR 128276 [11] silently drops every HWASan and MTE stack-tagging scope marker during LTO stream-in. Figure 3(b) shows that **50 bugs (45.0%) have High security severity** and 39 (35.1%) have Medium severity. These flaws bypass CFI landing pads, corrupt shadow stacks, or disable memory sanitizers without emitting any warnings. Portable cross-ISA flaws account for 42.3% (47/111) of the total. Among the 64 architecture-restricted bugs, **54.7% (35/64) affect only non-x86 backends** (AArch64, RISC-V, and LoongArch64), such as broken RISC-V `Zicfilp` landing pads [12], premature RVV stack-clash CFA switches [13], LoongArch canary leaks [14], and AArch64 PAC/KCFI flaws [15], [16]. These results show that newer non-x86 defenses harbor dense silent failure modes missed by x86-only testing.

```c
// Compile: gcc -O2 -mshstk
void test_cet(long *cnt, void *tok) {
  long v = *cnt - 1; *cnt = v;
  _clrssbsy(tok); // clears hardware ZF = 0
  if (v == 0) on_zero();
}
// Emitted x86-64 assembly:
// subq $1, (%rdi) # sets ZF when *cnt == 0
// clrssbsy (%rsi) # hardware clears ZF = 0
// je on_zero # testq deleted; dead!
```

Listing 6. Case study 1 (GCC PR 128271, fixed in GCC 14–17): missing RTL clobbers on x86 CET shadow-stack intrinsics.

```c
// Flags: -O2 -march=armv8-a+sve -fsanitize=safe-
    stack
#include <arm_sve.h>
void sink(svint32_t *p);
void f(void) {
  svint32_t v = svdup_s32(-1); // width = VL bytes
  sink(&v); // escapes to unsafe
}
// Emitted AArch64 assembly:
// sub x9, x21, #16 // fixed 16-B unsafe slot
// str x9, [x20, x19] // updates unsafe_sp
// st1w {z0.s}, p0, [x21, x8, lsl #2] // VL B!
```

Listing 7. Case study 2 (LLVM #237864, confirmed): AArch64 SafeStack allocates a fixed 16-byte slot for scalable SVE locals.

**Case Study 1: Silent CET Shadow-Stack Corruption in GCC.** Listing 6 shows a high-severity x86 CET shadow-stack bug found by DEFENZZ (GCC PR 128271 [17]). Per the Intel SDM, `CLRSSBSY` clears a shadow-stack busy token, writes `CF`, and unconditionally clears `ZF` to zero. In addition, `WRSSQ` writes shadow-stack memory that ordinary `MOV` loads can read. From this specification, DEFENZZ extracted an invariant requiring shadow-stack intrinsics to preserve subsequent flag-dependent branches and post-write verification reads. In `gcc/config/i386/i386.md`, however, five CET patterns (`clrssbsy`, `@wrss<mode>`, `@wruss<mode>`, `saveprevssp`, and `rstorssp`) omitted `FLAGS_REG` and memory clobbers. As a result, RTL `compare-elim` deleted `testq` after `_clrssbsy` and reused stale flags from `subq`, making `je on_zero` permanently dead. In the same pass pipeline, RTL CSE folded post-`_wrssq` verification checks (`*ss == val`) to zero. GCC maintainers patched trunk within 24 hours and backported the fix to GCC 16, 15, and 14.

**Case Study 2: AArch64 SafeStack Frame Overflow in LLVM.** Listing 7 shows a high-severity flaw between SafeStack and ARM SVE in Clang 23.1.2 (LLVM #237864 [18]). Under `-fsanitize=safe-stack`, escaping stack buffers move to an unsafe stack to isolate return addresses on the safe stack. Guided by mapped branches in `SafeStack.cpp`,

TABLE II
SUMMARY OF 111 PREVIOUSLY UNKNOWN SILENT DEFENSE BUGS DISCOVERED BY DEFENZZ ACROSS GCC AND LLVM.

| Defense Family | Checkers (G/L) | GCC | LLVM | Total | Conf. (Fix) | Affected ISAs | Representative Upstream IDs |
|---|---|---|---|---|---|---|---|
| CFI & Landing Pads | 201 (39/162) | 9 | 22 | 31 | 11 (3) | x86, ARM64, RV64, LA64 | PR 125084, 128360; #208751, #236673 |
| Stack & Shadow Stacks | 113 (62/51) | 12 | 11 | 23 | 6 (3) | x86, ARM64, RV64, LA64, MIPS | PR 125049, 128271; #237864, #237865 |
| Memory Sanitizers & MTE | 179 (58/121) | 8 | 12 | 20 | 3 (0) | x86, ARM64, RV64, LA64 | PR 128276, 128280; #237909, #236685 |
| Scrubbing & Speculation | 121 (73/48) | 13 | 7 | 20 | 5 (1) | x86, ARM64, RV64, Generic | PR 128287, 128388; #236795, #240257 |
| Bounds & Fortification | 130 (100/30) | 12 | 0 | 12 | 3 (2) | x86, ARM64, RV64, LA64 | PR 125079, 128591, 128592 |
| Call, Link & Enclave | 171 (142/29) | 4 | 1 | 5 | 2 (2) | x86, RV64, Generic | PR 128015, 128283; #220866 |
| **DEFENZZ Total** | **915 (474/441)** | **58** | **53** | **111** | **30 (11)** | **All evaluated ISAs** | **All 111 filed upstream** |
| Csmith [2] (hardened) | – | 0 | 0 | 0 | 0 (0) | x86, ARM64, RV64, LA64 | **[TBD: N]** crashes; 0 silent bugs |
| YARPGen [3] (hardened) | – | 0 | 0 | 0 | 0 (0) | x86, ARM64, RV64, LA64 | **[TBD: N]** crashes; 0 silent bugs |

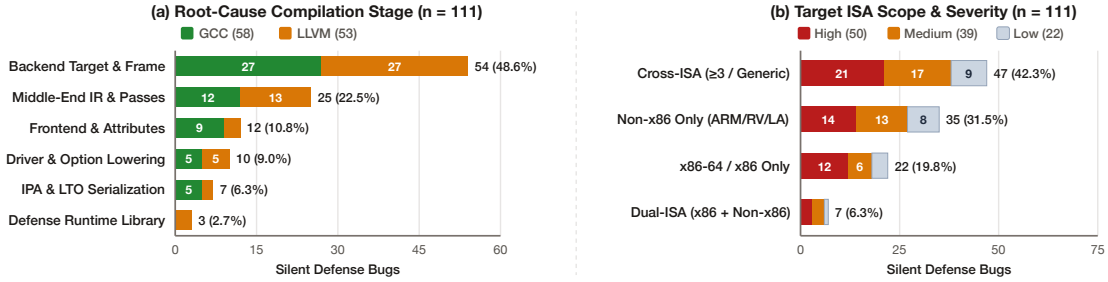



Fig. 3. Characterization of the 111 silent defense bugs discovered by DEFENZZ: (a) distribution across compiler stages in GCC and LLVM, and (b) target ISA scope broken down by security severity.

the Generator Agent synthesized a test program passing an escaping `svint32_t` local to an opaque `sink`. Because `getStaticAllocaAllocationSize()` returned zero for the scalable vector type, `moveStaticAllocasToUnsafeStack()` turned that zero size into a fixed 16-byte slot (`sub x9, x21, #16`), after which `st1w` wrote `VL` bytes to `usp - 16`. Running the binary under `qemu-aarch64` with `VL = 32` and `64` bytes overwrote 16 and 48 bytes of caller unsafe-stack data without any warning. Compiling without `-fsanitize=safe-stack` sized the frame properly via `addvl`, and LLVM maintainers confirmed the miscompilation.

### C. RQ2: Invariant Extraction and Formal Oracle Quality

**Invariant Extraction and RAG Grounding.** Table III tracks how DEFENZZ builds and validates its oracle catalog offline. In Step 1, segmented CoT analyzed 580 modular pass, target-hook, and manual slices (6.16M characters) across GCC and LLVM, extracting 904 security invariants. Hybrid RAG distilled 100 historical defense fixes into mechanism-neutral queries and retrieved 1,554 candidate slices. Origin-anchor exclusion and source grounding rejected 1,537 noisy or duplicate hits, accepting 17 cross-defense invariants with zero overlap against CoT. Step 1 produced **921 candidate invariants** (475 GCC, 446 LLVM), grounded in 671 compiler assertions or hooks and 250 specification rules. In Step 2, the mapping agent resolved all 921 invariants to **2,922** pass–function–branch paths across 747 source files with zero failures.

TABLE III
OFFLINE INVARIANT EXTRACTION, SOURCE MAPPING, AND TWO-GATE CHECK IR VALIDATION FUNNEL ACROSS GCC AND LLVM.

| Offline Pipeline Metric | GCC | LLVM | Total |
|---|---|---|---|
| Modular source/manual slices reviewed | 286 | 294 | 580 |
| Segmented CoT invariants accepted | 465 | 439 | 904 |
| Hybrid RAG candidates retrieved | 785 | 769 | 1,554 |
| Hybrid RAG invariants accepted (0 overlap) | 10 | 7 | 17 |
| **Step 1 Candidate Invariant Catalog** | **475** | **446** | **921** |
| Mapped pass–function–branch paths | 1,669 | 1,253 | 2,922 |
| Instrumented source files (`GCC/CLANG_INST`) | 342 | 405 | 747 |
| First-attempt Check IR pass rate | 84.0% | 80.5% | 82.3% |
| Gate 1 errors caught (well-formedness) | 36 | 80 | 116 |
| Gate 2 errors caught (differential anchoring) | 48 | 41 | 89 |
| Quarantined rules after 3 attempts | 1 | 5 | 6 |
| **Step 3 Validated Oracles Entering Fuzzing** | **474** | **441** | **915** |

**Two-Gate Check IR Validation Funnel.** In Step 3, DEFENZZ formalized all 921 invariants into Check IR across 1,120 bounded authoring attempts (1.22 attempts per invariant). On the first attempt, **758 rules (82.3%) passed both deterministic validation gates** without human intervention. Across failed attempts, Gate 1 (formal well-formedness) caught 116 errors: 55 hallucinated primitives outside $\Omega$, 55 malformed predicate expressions, and 6 unbound fields in $\mathcal{T}_i$. Gate 2 (differential baseline anchoring) caught another 89 errors, comprising 81 probe misfires between hardened and unhardened builds and

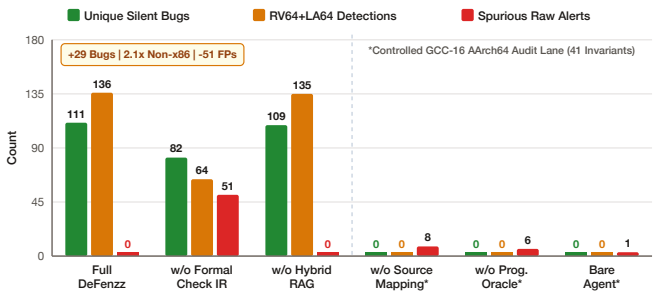



Fig. 4. Ablation comparison across DEFENZZ variants showing unique silent defense bugs, non-x86 (RISC-V and LoongArch64) detections, and spurious raw alerts.

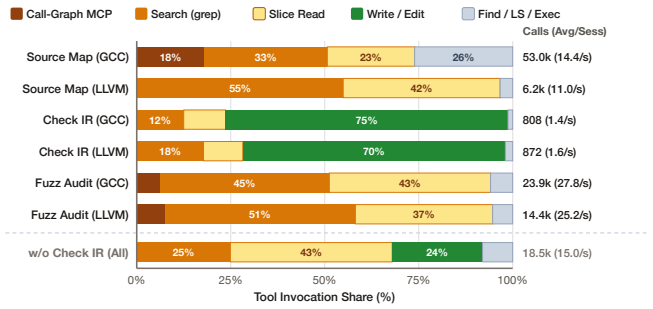



Fig. 5. Distribution and volume of agent MCP and workspace tool invocations across pipeline stages in DEFENZZ versus raw Go checker authoring (*w/o Check IR*).

TABLE IV
CONTROLLED ABLATION RESULTS ACROSS DEFENZZ VARIANTS. UNRUN COMPONENT ABLATIONS ARE MARKED AS **TBD**.

| System Variant | 1st-Pass | Tok. | RV+LA | FPs | Bugs |
|---|---|---|---|---|---|
| **Full DEFENZZ (Check IR)** | **82.3%** | **54.8M** | **136** | **0** | **111** |
| w/o Formal Check IR | 70.5% | 638.4M | 64 | 51 | 82 |
| w/o Hybrid RAG | TBD | TBD | TBD | TBD | TBD |
| w/o Source Mapping | TBD | TBD | TBD | TBD | TBD |
| w/o Prog. Oracle | TBD | TBD | TBD | TBD | TBD |
| Bare Agent | TBD | TBD | TBD | TBD | TBD |

8 Go lowering violations. Exact gate diagnostics enabled the authoring agent to repair 157 of the 163 failed rules within three attempts, quarantining only 6 rules. In total, **915 validated programmatic oracles (99.3%)** entered online fuzzing, implemented as 903 static phase-1 checkers and 12 sandboxed dynamic phase-2 checkers.

### D. RQ3: Ablation Study

**Impact of Formal Check IR Synthesis.** Figure 4 and Table IV compare full DEFENZZ against *w/o Formal Check IR*, where the agent writes standalone Go checkers directly from invariants. Across all 921 invariants, raw Go synthesis lowers first-attempt validation from 82.3% to 70.5%. It also triggers 98 Go compile or type errors versus 7 lowering errors under Check IR (**14.0× fewer**). Because raw Go agents repeatedly debug multi-hundred-line parsers, Step 3 token usage rises from 54.81M to 638.44M (**11.6**×) and authoring time grows from 2.20 h to 17.11 h (**7.8**×).

To isolate oracle quality from seed-generation randomness, we ran both checker suites on the same candidate triggers across all four ISAs. Check IR lifts total ISA-level detections from 188 to 315 (**1.68**×). Non-x86 backends benefit most: detections grow from 33 to 72 on RISC-V (**2.18**×) and from 31 to 64 on LoongArch64 (**2.06**×), reaching 136 versus 64 combined (**2.13**×). Among 104 triggers missed by raw Go checkers, 80 returned **NA** because hand-generated parsers hardcoded x86-64 or AArch64 mnemonics and bailed out on RISC-V or LoongArch64. Check IR avoids this gap by reading backend conventions directly from $\mathcal{T}_i$. Check IR also eliminates **51 spurious raw FAIL alerts** (12 GCC, 39 LLVM) caused by matching mnemonics inside assembly comments or inverting signed frame offsets. In total, Check IR uniquely uncovers **29 verified silent defense bugs** (11 GCC, 18 LLVM, including 11 High-severity bugs), lifting total bug discovery from 82 to 111.

**Remaining Component Ablations. [TBD: Experiments for *w/o Hybrid RAG*, *w/o Source Mapping*, *w/o Programmatic Oracle*, and *Bare Agent* are reserved for completion. This paragraph will report invariant recall without RAG transfer, branch coverage without `DEFENZZ_HIT` probes, false positives under LLM adjudication, and stalls in unconstrained agents.]**

### E. RQ4: Agent Telemetry, Token Usage, and Overhead

**Agent Tool-Call Telemetry.** Figure 5 breaks down 99,236 tool calls across 6,798 agent sessions in DEFENZZ, plus 18,495 calls across 1,229 sessions in *w/o Check IR*. Tool patterns mirror the structure of each compiler. In GCC source mapping, agents issue 52,953 calls across 3,677 sessions (14.4 calls/session). Absent in LLVM, 18.0% of GCC calls query the Call-Graph MCP server to link flags across GIMPLE passes, RTL expanders, target hooks, and `.md` files. In LLVM, modular C++ pass files let agents map all 446 invariants in 6,244 calls (11.0 calls/session), using lexical search (54.8%) and slice reads (41.7%). In Step 3, Check IR authoring requires only **1,680 tool calls** across both compilers (1.4–1.6 calls/session), with over 70% directly writing concise specification files. Without Check IR, raw Go authoring requires **18,495 tool calls** (15.0 calls/session, **11.0× higher**) to search and inspect parser APIs. During online fuzzing and auditing, agents average 25.2–27.8 read-only navigation calls per session alongside 705 deterministic gRPC oracle runs.

**Token Efficiency, Build Overhead, and Runtime.** Table V reports LLM token usage and wall-clock time across all stages. One-time offline preparation (Steps 1–3) takes 709.17M tokens and 34.52 h, led by GCC source mapping (28.23 h versus 1.27 h on LLVM). Both the mapped branch hierarchies and compiled Check IR checkers are reusable across fuzzing runs and point releases. Deterministic AST instrumentation (Step 2b) rewrites 747 files with zero LLM calls. Building

TABLE V
STAGE-BY-STAGE LLM API INVOCATIONS, TOKEN ACCOUNTING, PREFIX KV-CACHE HIT RATES, AND WALL-CLOCK RUNTIME ACROSS THE FULL GCC AND LLVM EVALUATION CAMPAIGNS.

| **Pipeline Stage** | **Phase** | **LLM Calls** | **Input (M)** | **Cached (Hit %)** | **Output (M)** | **Total (M)** | **Time (h)** |
|---|---|---|---|---|---|---|---|
| Step 1: Security Invariant Extraction | Offline | –[†] | – | – | – | 15.00 | 2.00 |
| Step 2a: Compiler Source Mapping | Offline | 35,523 | 617.07 | 572.84 (92.8%) | 22.28 | 639.36 | 29.50 |
| Step 2b: AST Branch Instrumentation | Offline | 0 | 0.00 | – | 0.00 | 0.00 | 0.82 |
| Step 3: Check IR Oracle Synthesis | Offline | 2,733 | 47.35 | 36.37 (76.8%) | 7.46 | 54.81 | 2.20 |
| Step 4: Directed Fuzzing & Audit | Online | 21,415 | 881.17 | 818.21 (92.9%) | 19.67 | 900.84 | 10.16 |
| **Full Campaign Total (GCC + LLVM)** | **All** | **59,671** | **1,545.59** | **1,427.42 (92.4%)** | **49.41** | **1,610.01** | **44.68** |

[†]Step 1 ran on pre-sliced source and manual batches under a 15.00M-token cap (10.00M GCC, 5.00M LLVM); per-call cache counters are recorded across Steps 2–4.

the instrumented compilers on 16 ARM Neoverse-N2 vCPUs takes 4.6 min (277.1 s) for `GCC_INST` and 25.8 min (1,550.8 s) for `CLANG_INST`. Because `DEFENZZ_HIT` probes stay dormant unless `DEFENZZ_LOG` is set, compiled target binaries incur zero runtime overhead. Online directed fuzzing (Step 4) completes in **10.16 h** (7.43 h GCC, 2.73 h LLVM) and consumes 900.84M tokens. Step 4 reaches its first verified silent defense bug after 31.9 min on GCC and 157.6 min on LLVM. Keeping invariant schemas, $\Omega$ signatures, and $\mathcal{T}_i$ fixed in prompt prefixes achieves a **92.4% overall KV-cache hit rate** (1,427.42M cached input tokens). As a result, online fuzzing costs an average of **5.5 min of wall-clock time and 8.12M tokens per discovered silent defense bug**.

## REFERENCES


[1] MITRE, "CVE-2023-4039: GCC AArch64 stack protector bypass with variable-length arrays," https://nvd.nist.gov/vuln/detail/CVE-2023-4039, 2023.

[2] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[3] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.

[4] GCC Bugzilla, "Bug 125079: tree-ssa-strlen get_string_length: __strcat_chk to __stpcpy_chk drops bound subtraction," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125079, 2026.

[5] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[8] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[9] GCC Bugzilla, "Bug 96191: aarch64 stack_protect_test canary leak," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191, 2020.

[10] J. Regehr *et al.*, "C-Reduce: A C and C++ program reducer," https://github.com/csmith-project/creduce, 2012.

[11] GCC Bugzilla, "Bug 128276: LTO stream-in drops HWASan/MTE stack scope markers," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=128276, 2026.

[12] ——, "Bug 128011: RISC-V Zicfilp emits patchable-function-entry NOPs before landing pad," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=128011, 2026.

[13] ——, "Bug 127318: RISC-V RVV stack-clash protection switches CFA register to t3 prematurely," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=127318, 2026.

[14] ——, "Bug 125049: LoongArch stack_protect_test leaves canary value in caller-visible register," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125049, 2026.

[15] LLVM GitHub Issues, "Issue 236673: [AArch64][KCFI] LowerKCFI_CHECK clobbers live argument register w9 under preserve_none," https://github.com/llvm/llvm-project/issues/236673, 2026.

[16] ——, "Issue 237862: [AArch64][PAC] Function pointer in data-key __ptrauth slot authenticated with IA key," https://github.com/llvm/llvm-project/issues/237862, 2026.

[17] GCC Bugzilla, "Bug 128271: [x86] CET shadow-stack clrssbsy/wrss* patterns omit FLAGS_REG and memory clobbers," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=128271, 2026.

[18] LLVM GitHub Issues, "Issue 237864: [SafeStack][AArch64] Scalable SVE local moved to unsafe stack gets fixed 16-byte slot," https://github.com/llvm/llvm-project/issues/237864, 2026.